

CSCE 627 THEORY OF COMPUTABILITY · MODULE 02 · LECTURE NOTES

Finite Memory

What a machine with no memory of how it got here cannot do.

MODULE OUTCOMES

- Define a DFA and an NFA and convert between them.
- Explain why non-determinism adds no power here.
- Prove a language non-regular with the pumping lemma.
- State the closure properties and use them in proofs.
- Recognise regular languages in practical settings.

1 The model, and non-determinism

A FINITE AUTOMATON'S ONLY MEMORY IS WHICH STATE IT IS IN

A finite set of states, a transition function on each input symbol, a start state, and a set of accepting states. The input is read once, left to right, with no ability to go back.

So the machine's entire memory of everything it has read is one state out of finitely many — which means **two different inputs that lead to the same state are indistinguishable from that point onward, for every possible continuation.**

That single observation is the whole theory. A language is regular exactly when it has finitely many distinguishable prefix classes — the Myhill–Nerode theorem — where two prefixes are distinguishable if some suffix puts one in the language and the other out. **Every non-regularity proof in this module is that fact in disguise.**

And it gives the minimal DFA for free: the states *are* the equivalence classes, so **the minimal automaton for a regular language is unique up to renaming of states** — which is an unusual property (nothing like it holds for context-free grammars or for Turing machines) and is what makes DFA minimisation a well-defined operation with a canonical answer.

AN NFA may have several transitions on one symbol, or none, or epsilon-transitions. It accepts if SOME path accepts.

CLAIM: every NFA has an equivalent DFA.

THE SUBSET CONSTRUCTION:

- the DFA's states are SETS of NFA states
- the DFA's state after reading w is exactly the set of NFA states reachable from the start on w
- accept if that set contains any NFA accepting state

Determinism is recovered by tracking ALL the places the NFA could simultaneously be.

THE COST: 2^n states in the worst case, and the bound is TIGHT -- there are languages for which every equivalent DFA genuinely needs exponentially many states.

SO NON-DETERMINISM BUYS CONCISENESS, NOT POWER. And that distinction recurs: in CSCE 637, whether non-determinism buys POWER for polynomial time is the P vs NP question, and it is open. Here the same question is settled, and the answer is no.

AND THIS IS WHY REGEX ENGINES COME IN TWO KINDS: Thompson NFA simulation (linear time, because it tracks a SET of states) and backtracking (exponential worst case, but it supports backreferences, which are not regular at all).

The regex-engine connection is the practical payoff of this section. Catastrophic backtracking is a direct consequence of leaving the regular languages: a pattern with a backreference is not describable by any finite automaton, so no linear-time simulation exists for it, so the engine must search — and the search is exponential. **Which means the choice between the two engine designs is a choice about which language class you are willing to support**, and the performance follows from the theory rather than from implementation quality.

2 Proving a language non-regular

THE PUMPING LEMMA IS THE PIGEONHOLE PRINCIPLE ON STATES

If a DFA with p states accepts a string of length at least p , then some state must repeat during the computation — there are more steps than there are states, so by the pigeonhole principle two of them coincide.

The segment of input consumed between the two visits to that state forms a loop, so it can be traversed zero times, once, or any number of times, and the machine ends in the same state and still accepts.

So every sufficiently long accepted string splits as xyz with $|y| > 0$ and $|xy| \leq p$, and xy^iz is accepted for every $i \geq 0$. The $|xy| \leq p$ condition records that the repeat happens within the first p symbols.

To prove a language non-regular: assume a pumping length p exists, choose a string in the language cleverly, and show that no valid split of it survives pumping. The art is entirely in the choice of string, and §2's worked example shows what a good choice does.

CLAIM: $L = \{ a^n b^n : n \geq 0 \}$ is not regular.

Suppose it is, with pumping length p .

Choose $s = a^p b^p$, which is in L and has length $\geq p$.

The lemma gives $s = xyz$ with $|y| > 0$ and $|xy| \leq p$.

Since $|xy| \leq p$, the prefix xy lies entirely within the a 's -- so y consists only of a 's, and y is non-empty.

Then $xyyz$ has strictly MORE a 's than b 's, so it is not in L .

But the lemma says $xyyz$ must be in L . Contradiction.

Therefore L is not regular.

WHY THE CHOICE OF s MATTERED: with $s = (ab)^p$ the split could straddle an a and a b , and the case analysis is harder. With $s = a^p b^p$ the $|xy| \leq p$ condition PINS y inside the a 's, which makes the contradiction immediate.

THE GENERAL TACTIC: choose s so that the $|xy| \leq p$ condition FORCES y into a region where pumping visibly breaks a counting or matching requirement.

AND HAVE THE INTUITION FIRST: $a^n b^n$ requires UNBOUNDED counting, and a machine with finite state cannot count without a bound. The lemma is the formal version of that observation -- so form the intuition, then write the proof.

3 Closure properties

Operation	Construction	Note
Union and intersection	The product automaton.	States are pairs (p, q) , one from each machine; accept according to the operation. Simulates both at once.
Complement	Swap accepting and non-accepting states — of a DFA.	This does NOT work on an NFA, because an NFA accepts if <i>some</i> path accepts and swapping does not negate that. A classic and costly error: convert to a DFA first.
Concatenation and star	Easy with an NFA — add ϵ -transitions.	Which is precisely what NFAs are for: these constructions are awkward on a DFA and trivial on an NFA.
Reversal, homomorphism, inverse homomorphism	Reverse all arrows and swap start and accepting; relabel symbols.	All straightforward, and all occasionally useful in proofs.
Intersection with a regular language	The product construction again.	The most useful proof tool in the module — see the callout.

WHY CLOSURE PROOFS BEAT PUMPING PROOFS WHEN THEY APPLY

Pumping arguments require inventing a suitable string and then checking every valid split of it. They are error-prone, the cases multiply when the string is not well chosen, and a student's first attempt usually picks a string that admits an awkward split.

Closure arguments reuse a result you already have. To show L is not regular, intersect it with a regular language and obtain a language already known not to be regular — if L were regular, the intersection would be too, by the table above, and it is not.

Example: the language of strings over $\{a, b\}$ with equally many a 's and b 's, intersected with the regular language a^*b^* , is exactly $\{a^n b^n\}$, which §2 settled. **Two lines, no case analysis.**

So try a closure argument first. And note that the pumping lemma is a necessary condition and not a sufficient one — there exist non-regular languages that satisfy it — **so a failed pumping argument proves nothing at all**, which is a distinction worth being careful about since the lemma is usually taught as if it were a characterisation. **Myhill–Nerode (§1) is the actual characterisation**, and it is sometimes the easier route.

4 Regular languages in real systems

- **Lexical analysis. Programming language tokens are regular by deliberate design** — identifiers, numbers, operators, comments — **which is why a lexer is a DFA and runs in time linear in the input** (CSCE 605 Module 02). **The design decision came first and the efficiency followed.**
- **Protocol and input validation**, where finite state is exactly the right amount of power — and where an unbounded parser would be a liability rather than a feature, since it could accept inputs nobody intended.
- **Model checking of finite-state systems**, where the state space is finite by construction, so everything about it is decidable and the techniques of this module apply directly.
- **Text search. Thompson NFA simulation is linear in the input and immune to catastrophic backtracking**, because it tracks the set of reachable states rather than searching paths — **which is why grep does not hang on adversarial patterns and several widely-used regex libraries do.**
- **And not: matching nested brackets, parsing HTML, or anything requiring balance. That is Module 03, and the well-known refusal to parse HTML with a regular expression is this theorem rather than an aesthetic preference.** Note also that 'regex' in most libraries is not regular — backreferences and lookahead strictly exceed the regular languages, which is exactly where the exponential behaviour comes from.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Sipser — chapter 1	Finite automata, the subset construction, and the pumping lemma — the reference for this module.
MIT 18.404J — lectures 1–3 (free)	The same material lectured , with problem sets and solutions.
Cox — Regular Expression Matching Can Be Simple And Fast (free)	\$1's engine distinction, in practice — Thompson simulation against backtracking, with measurements. Required reading for anyone who uses regular expressions.
Hopcroft, Motwani & Ullman — chapters 2–4	Myhill–Nerode and DFA minimisation , treated more thoroughly than Sipser does.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Implement a DFA simulator** and a DFA for a language of your choosing.
- 2 **Implement the subset construction** and convert an NFA with 8 states. Report the DFA's size.
- 3 **Construct a language where the blow-up is exponential** and confirm it.

- 4 **Prove three languages non-regular** by the pumping lemma, choosing the strings yourself.
- 5 **Deliberately choose a bad string** for one of them and report how much harder the case analysis becomes.
- 6 **Prove one of the same languages non-regular by a closure argument** and compare the two proofs' lengths.
- 7 **Complement an NFA by swapping accepting states** and exhibit a string showing the result is wrong.
- 8 **Minimise a DFA** and relate its states to Myhill–Nerode classes.
- 9 **Write a catastrophically backtracking regex** and time it against input length.
- 10 **Run the same pattern against a Thompson-based engine** (RE2, or Go's regexp) and compare.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What is a finite automaton's entire memory, and what follows?
- 2 State Myhill–Nerode and explain why the minimal DFA is unique.
- 3 Describe the subset construction and its cost.
- 4 What does non-determinism buy here, and what is the open analogue?
- 5 Derive the pumping lemma from the pigeonhole principle.
- 6 Prove $a^n b^n$ non-regular, and say why the string choice mattered.
- 7 Why is the pumping lemma not a characterisation?
- 8 Name five closure properties and the one that fails on NFAs.
- 9 Why is the intersection trick the better proof technique?
- 10 Why do some regex engines backtrack catastrophically?